

Examen réparti 2

Master 1 SAR

UE MU4IN401

L. Arantes, P. Sens, J. Sopena,

Janvier 2021

2 heures – **Tout document papier autorisé**

Tout appareil électronique interdit

Le barème est donné à titre indicatif

Exercice 1 – Système de fichiers (6,5 points)

On considère le fichier `/tmp/fich` qui contient la chaîne de caractère `"examen2021"` et est désigné par l'inode 20. Les inodes ont la même structure que celle vue en TD. Un programme fait les instructions suivantes :

```
1  int main() {
2      int fd1,fd2,fd3;
3      char buf[4];
4      fd1 = open("/tmp/fich", O_RDWR);
5      write(fd1,"AAA",3);
6      if (fork()==0) {
7          fd2 = open("/tmp/fich", O_RDONLY);
8          read(fd2,buf,3);
9          buf[3] = 0;
10         printf("%s\n", buf);
11         write(fd1,"BBB",3);
12         exit(0);
13     }
14     fd3 = open("/tmp/fich", O_WRONLY);
15     wait(NULL);
16     return 0;
17 }
```

Question 1 1 point

Quel est l'affichage fait par ce programme ? Quel est le contenu du fichier `/tmp/fich` à la fin de l'exécution ?

Solution:

Affichage : AAA

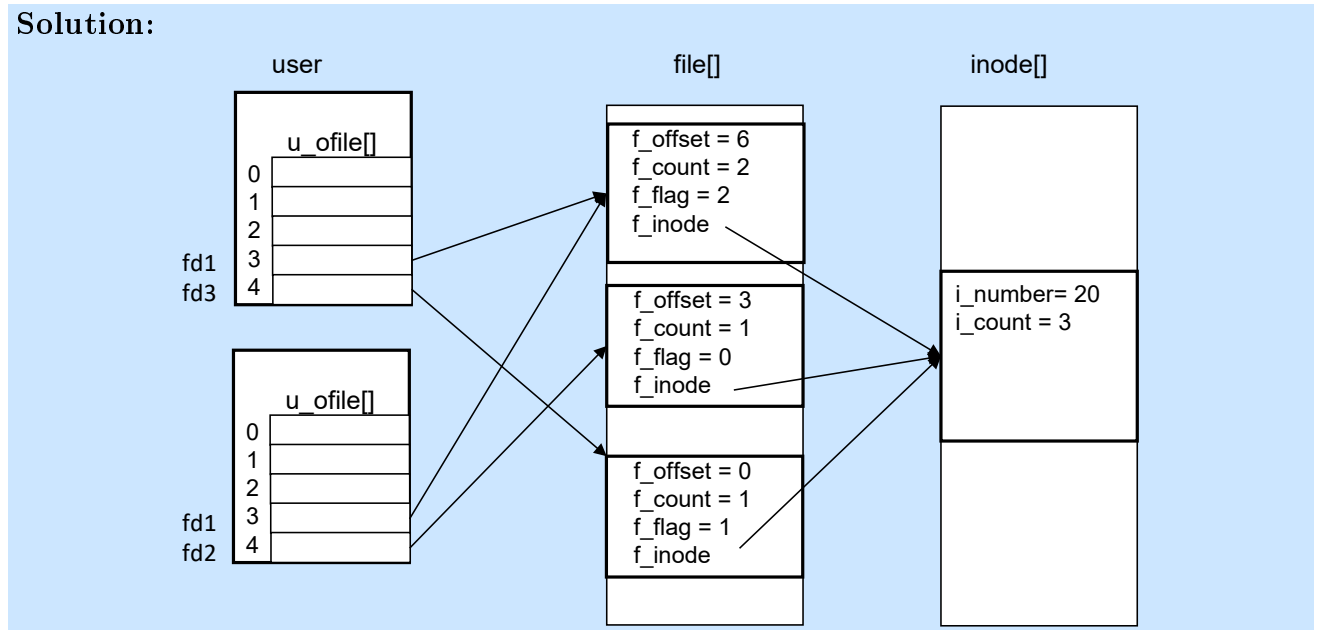
Contenu : AAABBB2021

Question 2 1½ points

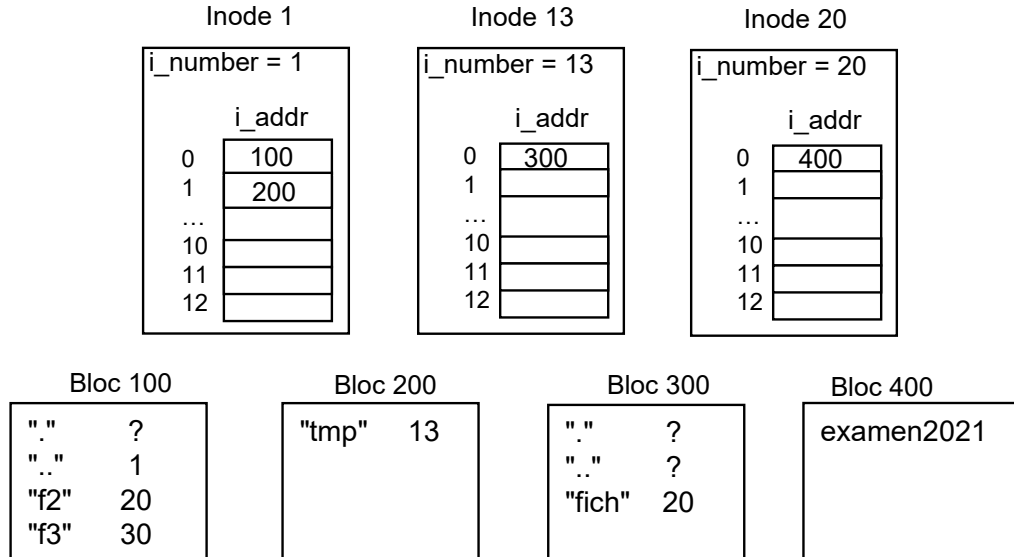
Donnez l'état, juste après la ligne ?? et en supposant que la ligne ?? a pu être exécutée, des

tables de descripteurs de fichiers (*u_ofile*), fichiers ouverts (*file*), et inodes (*inode*). Faites un schéma en pensant à représenter les champs *f_count*, *f_offset*, *f_flags*, *i_count*, *i_number* et *i_size*. Pour les inodes vous représenterez juste l'état de l'inode 20.

Solution:



On considère la partition numéro 0 avec des blocs de 512 octets. Le superbloc correspond au bloc 1. La table des inodes commence à partir du bloc 2 inclus. Les inodes sont numérotées à partir de 1. Les inodes ont une taille de 32 octets. La racine est désignée par l'inode 1. Nous considérons les inodes et les blocs suivants (les '?' désignent des informations manquantes) :



Question 3 $\frac{1}{2}$ point

Complétez le contenu des blocs 100 et 300 en justifiant vos réponses.

Solution:

Bloc 100 :

"." 1 (l'inode du répertoire courant)

Bloc 300 :

"." 13 (l'inode du répertoire courant)

".." 1 (l'inode du répertoire parent)

Question 4 1 point

Dans quels blocs sont situés sur disque les inodes 1, 13 et 20 ? Donnez la formule qui à partir d'un numéro d'inode retourne le numéro de bloc sur disque.

Solution:

inode 1 : bloc 2

inode 13 : bloc 2

inode 20 : bloc 3

16 inodes par bloc, inode 1 = 1ere inode bloc = $(i_number + 31) / 16$ ou bloc = $2 + (i_number - 1) / 16$

Question 5 1/2 point

Est-ce le fichier "/f2" est un lien hard ou symbolique vers le fichier "/tmp/fich" ? Que ce passe-t-il si le fichier "/tmp/fich" est effacé (par la commande rm) et qu'on tente ensuite d'afficher le contenu du fichier "/f2" ? Justifiez vos réponses.

Solution:

C'est un lien hard car f2 désigne la même inode

Après la suppression, le contenu du fichier pourra être affiché car $i_nlink > 0$.

Question 6 2 points

On suppose que seule l'inode 1 (la racine) est initialement en **mémoire**. Quels sont, dans l'ordre, les blocs et inodes accédés par les lignes ?? et ?? de ce programme ?

Solution:

ligne ?? : inode 1, bloc 100, bloc 200, bloc 2, inode 13, bloc 300, bloc 3, inode 20

ligne ?? : inode 20, bloc 400

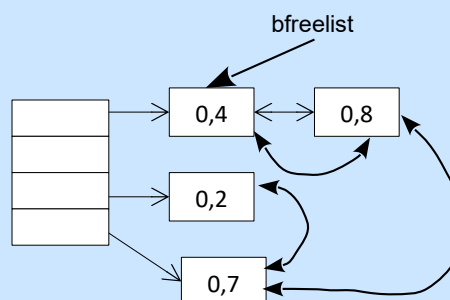
Exercice 2 – Buffer cache (5 points)

Nous considérons un buffer cache de **4 cases**. L'état de la bfreelist de la tête vers la queue est le suivant sous le format $\langle dev, bloc \rangle$: $\langle 0, 4 \rangle \langle 0, 8 \rangle \langle 0, 7 \rangle \langle 0, 2 \rangle$.

Question 1 1 point

Faites un schéma de la structure du buffer cache en considérant une table hbuf de taille 4 et la fonction de hachage $fh(dev, bloc) = (dev + bloc) \% 4$.

Solution:



Question 2 1 1/2 points

On suppose qu'aucun buffer n'a initialement le flag DEL_WRI. Le noyau exécute la séquence suivante :

```

1 struct buf *bp;
2 bp = bread(0,4);
3 brelse(bp);
4 bp=bread(0,9);
5 brelse(bp);
6 bp = bdwrite(0,7)
7 brelse(bp);

```

Donnez l'état de la bfreelist (la liste des blocs qui la compose de la tête vers la queue) après les lignes 3, 5 et 7 ainsi que le nombre d'entrées/sorties physiques générées par cette séquence.

Solution:

Init : $\langle 0, 4 \rangle \langle 0, 8 \rangle \langle 0, 7 \rangle \langle 0, 2 \rangle$
 ligne 3 $\langle 0, 4 \rangle$: $\langle 0, 8 \rangle \langle 0, 7 \rangle \langle 0, 2 \rangle \langle 0, 4 \rangle$
 ligne 5 $\langle 0, 9 \rangle$: $\langle 0, 7 \rangle \langle 0, 2 \rangle \langle 0, 4 \rangle \langle 0, 9 \rangle$, 1 E/S (bloc $\langle 0, 9 \rangle$)
 ligne 7 $\langle 0, 7 \rangle$: $\langle 0, 2 \rangle \langle 0, 4 \rangle \langle 0, 9 \rangle \langle 0, 7 \rangle$
 Total : 1 E/S

Question 3 1½ points

On remplace le code par la séquence suivante :

```

1 struct buf *bp;
2 bp = breada(0,4,9);
3 brelse(bp);
4 bp = bdwrite(0,7)
5 brelse(bp);

```

A quel moment et par quelle fonction, le buffer contenant le bloc $\langle 0, 9 \rangle$ sera inséré dans la bfreelist par un appel à brelse ? Quel est maintenant l'état de la bfreelist après la ligne 3.

Solution:

Le bloc anticipé est inséré dans la bfreelist en fin d'E/S par la fonction iodone.
 ligne 3 $\langle 0, 4, 9 \rangle$: $\langle 0, 7 \rangle \langle 0, 2 \rangle \langle 0, 4 \rangle$

Question 4 1 point

Dans le code de la question 2 (i.e., sans le breada), on suppose maintenant qu'initialement le buffer du bloc $\langle 0, 8 \rangle$ a le flag DEL_WRI positionné. Quel est l'état de la bfreelist après la ligne 5. Y-a-t-il à cet instant des buffers qui ne sont pas dans la bfreelist, si oui le(s) quel(s) ?

Solution:

ligne 5 $\langle 0, 9 \rangle$: $\langle 0, 2 \rangle \langle 0, 4 \rangle \langle 0, 9 \rangle$, 1 E/S (bloc $\langle 0, 9 \rangle$)
 Le bloc $\langle 0, 8 \rangle$ est présent mais plus dans la bfreelist car en cours de copie sur disque.

Exercice 3 – Appel système - Fichier (4 points)

On souhaite implémenter dans le noyau du TD les deux appels systèmes suivants.

- `void setseek(int val)`. Cet appel positionne l'offset (`f_offset`) à *val* pour tous les fichiers ouverts par le processus courant.
- `int readmaxseek(int fd)`. Cet appel retourne l'offset maximum utilisé par tous les processus manipulant le même fichier (inode) que celui référencé par *fd*.

Remarque : ces deux appels sont indépendants.

Question 1 2 points

Donnez le code dans le noyau de l'appel système `sys_setseek()` correspondant à `void setseek(int val)`.

Solution:

```
1 sys_setseek(){
2     int i;
3     for (i=3;i<NOFILE;i++)
4         if (u.u_ofile[i] != NULL)
5             u.u_ofile[i]->f_offset = u.u_arg[0];
6     return;
7 }
```

Question 2 2 points

Donnez le code dans le noyau de l'appel système `sys_readmaxseek()` correspondant à `int readmaxseek(int fd)`.

Solution:

```
1 sys_readmaxseek(){
2     struct file *fp;
3     struct inode *ip = u.u_ofile[u.uarg[0]]->f_inode;
4     int max = 0;
5     for (fp=&file[0];fp<file[NFILE];fp++)
6         if (fp != NULL && fp->f_inode == ip && fp->f_offset > max )
7             max = fp->f_offset;
8     u.u_ar0[R0] = max;
9     return;
10 }
```

Exercice 4 – Appel système - Processus (5,5 points)

Nous considérons qu'un processus ne peut créer qu'au plus 1 processus fils et que tous les processus se terminent correctement. Par conséquent, il est possible d'avoir des chaînes de processus où le processus initial (celui du main) crée un processus qui à son tour crée un second processus et ainsi de suite jusqu'à un processus qui ne crée pas de processus comme montre la Figure ??.

Nous voulons offrir la fonction `int number_ancestors (int pid_shell)` qui retourne le nombre d'ancêtres (père, grand-père etc.) du processus sans considérer le processus shell qui a lancé le programme ni le processus init comme ancêtre. Par exemple, le programme ci-dessous crée une chaîne de N=4 processus en générant l'arborescence de la Figure 1. L'appel de la fonction `number_ancestors` par les processus père, fils, petit-fils et arrière petit-fils retourne 0, 1, 2 et 3 respectivement.

```

1  #define N 4
2  int main (int argc, char ** argv ) {
3      int nb, i =1;
4      int pid_shell= getppid(); /* le pid du shell qui a lancé le programme =
        le père du main */
5
6      while ((i < N) && (fork ( ) == 0) )
7          i++;
8
9      nb = number_ancestors (pid_shell);
10
11     if (i != N)
12         wait (NULL);
13
14     printf ("nombre_ancêtres", %d \n, nb) ;
15     return 0;
16 }

```

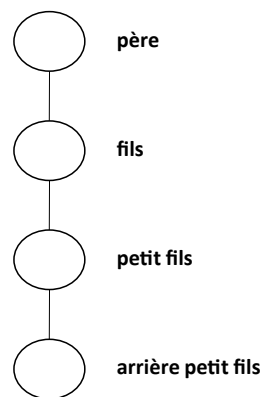


FIGURE 1 – Chaîne de processus créés par le programme

Question 1 2 points

Donnez le code dans le noyau de l'appel système `sys_number_ancestors ( )` correspondant à `int number_ancestors (int pid_shell)`.

Solution:

```

1  sys_number_ancestors ( )
2      register struct proc *p;
3      int pid_pere = u.u_procp->p_ppid;
4      u.u_ar0[R0] =0;
5
6      while ( (pid_pere != u.u_arg[0]) && (pid_pere != 1) ) {
7          u.u_ar0[R0] ++;
8          for (p = &proc[0]; p < &proc[NPROC]; p++) {
9              if (p->p_pid == pid_pere)
10                 pid_pere = p->p_pid;
11                 break;
12             }
13             if ( p == &proc[NPROC])
14                 return;
15         }
16     return;

```

Question 2 1/2 point

Si nous enlevons les lignes ?? et ?? du programme, est-ce que la fonction `number_ancestors`

pourrait renvoyer 0 à d'autres processus que le processus principal (père) ? Justifiez votre réponse.

Solution:

Oui. Par exemple, si le fils se termine avant le petit-fils, celui-ci sera hérité par init et lorsqu'il appelle `number_ancestors (int pid_shell)` la fonction rendra 0.

Nous voulons maintenant implémenter la fonction `void ancestor_bloc (int pid_shell)` qui doit bloquer l'appelant (1) s'il a créé un fils et (2) que le dernier processus de la chaîne (celui qui n'a pas de fils) n'a pas encore exécuté la fonction `ancestor_bloc`. S'il s'agit du dernier processus de la chaîne, la fonction doit réveiller tous les processus qui se sont endormis dans la fonction.

Nous considérons que :

1. Tous les processus de la chaîne appellent `ancestor_bloc`.
2. Un processus se bloque avec la priorité de réveil PWAIT sur le champ `p_flag` de sa propre entrée dans la table `proc` [].
3. Vous pouvez définir des drapeaux (pas besoin de donner leur valeur) et utiliser le champ `p_flag` de l'entrée du processus dans la table `proc` [] pour contrôler la synchronisation, si nécessaire. Le programme ci-dessous montre l'utilisation de la fonction `ancestor_bloc`. Le dernier processus de la chaîne (ici, l'arrière petit-fils) ne se bloque pas dans la fonction et celle-ci doit toujours se terminer avant celles appelées par ces ancêtres.

```
1 #define N 4
2 int main (int argc, char ** argv) {
3     int i = 1;
4
5     int pid_shell = getppid ();
6     while ((i < N) && (fork () == 0))
7         i++;
8
9     ancestors_block (pid_shell);
10    return 0;
11 }
```

Question 3 3 points

Donnez le code dans le noyau de l'appel système `sys_ancestors_bloc()` correspondant à `void ancestor_bloc (int pid_shell)`.

Solution:

```
1 #define A_BLOC    xx
2 #define A_NBLOC   xxx
3
4 sys_ancestors_bloc ( )
5     register struct proc *p;
6     int pid_pere;
7
8     for (p = &proc[0]; p < &proc[NPROC]; p++) {
9         if (p->p_ppid == u.u_procp->p_pid) {
10             /* processus appelant est le père du processus p */
11             if ( ! (u.u_procp -> p_flag & A_NBLOC)) {
12                 /* processus se bloque parce que le dernier
13                  * processus de la chaîne n'a pas encore exécuté
14                  * ancestors_bloc ( ) */
15                 u.u_procp->p_flag |= A_BLOC;
16                 sleep (&(u.u_procp->p_flag), PWAIT);
17             }
18         }
19     }
```

```

16         return;
17     }
18     } /* for */
19     /* pas de fils - dernier processus de la chaîne - chercher les ancê
      tres */
20     int pid_pere = u.u_procp->p_ppid;
21     while ( (pid_pere != u.u_arg[0]) && (pid_pere != 1) ) {
22         for (p = &proc[0]; p < &proc[NPROC]; p++) {
23             if (p->p_pid == pid_pere) {
24                 if (p->p_flag & A_BLOC) { /* ancêtre bloqué */
25                     p->p_flag &= A_BLOC;
26                     pid_pere = p->p_ppid;
27                     wakeup (&(p->p_flag));
28                 }
29                 p->p_flag |= A_NBLOC /* indiquer que le
                    dernier processus a exécuté */
30                 break;
31             }
32         } /* for */
33     } /* while */
34     return;
35 }

```

Annexe

```
1  #define NADDR 13
2
3  struct inode
4  {
5      char          i_flag;
6      char          i_count; /* reference count */
7      dev_t         i_dev; /* device where inode resides */
8      ino_t         i_number; /* i number, 1-to-1 with device address */
9      unsigned short i_mode;
10     short          i_nlink; /* directory entries */
11     short          i_uid; /* owner */
12     short          i_gid; /* group of owner */
13     off_t          i_size; /* size of file */
14     struct {
15         daddr_t    i_addr[NADDR]; /* if normal file/directory */
16         daddr_t    i_lastr; /* last logical block read (for read-
17                               ahead) */
18     };
19 };
20
21 struct inode inode[NINODE]; /* The inode table itself */
22
23 /* flags */
24 #define ILOCK 01 /* inode is locked */
25 #define IUPD 02 /* file has been modified */
26 #define IACC 04 /* inode access time to be updated */
27 #define IMOUNT 010 /* inode is mounted on */
28 #define IWANT 020 /* some process waiting on lock */
29 #define ITEXT 040 /* inode is pure text prototype */
30 #define ICHGO 100 /* inode has been changed */
31
32 /* modes */
33 #define IFMT 0170000 /* type of file */
34 #define IFDIR 0040000 /* directory */
35 #define IFCHR 0020000 /* character special */
36 #define IFBLK 0060000 /* block special */
37 #define IFREG 0100000 /* regular */
38 #define IFMPC 0030000 /* multiplexed char special */
39 #define IFMPB 0070000 /* multiplexed block special */
40 #define ISUID 04000 /* set user id on execution */
41 #define ISGID 02000 /* set group id on execution */
42 #define ISVTX 01000 /* save swapped text even after use */
43 #define IREAD 0400 /* read, write, execute permissions */
44 #define IWRITE 0200
45 #define IEXEC 0100
46
47 /*
48  * One file structure is allocated
49  * for each open/creat/pipe call.
50  * Main use is to hold the read/write
51  * pointer associated with each open
52  * file.
53  */
54 struct file
55 {
56     char          f_flag;
57     cnt_t         f_count; /* reference count */
58 }
```

```

12         int      *f_inode;      /* pointer to inode structure */
13         long      f_offset;      /* read/write character index */
14     } file[NFILE];
15
16
17     /* flags */
18     #define FOPEN      (-1)
19     #define FREAD      0x0001
20     #define FWRITE     0x0002
21     #define FPIPE      0x0004
22
23
24     /* open parameters */
25     #define O_RDONLY    0
26     #define O_WRONLY    1
27     #define O_RDWR      2

```